

FoldPipe: Bounded Remote Streaming of Native Molecular-Graph Shards

with Conditional I/O–Compute Overlap

Dhiren Khatri

Vivekanand Education Society’s Institute of Technology (VESIT)

<https://github.com/aviatorlf/FoldPipe>

Draft of August 17, 2026

Abstract

Training atomistic neural networks on ephemeral or memory-constrained accelerator instances can require retrieving preprocessed molecular graphs from remote storage. FoldPipe is a lightweight Python orchestration layer that iterates over native PyTorch/PyTorch Geometric shard files, retrieves one shard ahead in a background thread, and keeps a constant-number shard working set. The prefetch principle is established systems practice rather than a new scheduling algorithm; the contribution here is a compact molecular-data integration and a source-pinned empirical characterization of when its overlap does and does not translate into wall-clock improvement.

We evaluate a SchNet energy-and-force training step on MD17 aspirin data using 20 paired, order-alternating passes on a Kaggle Tesla T4. Each pipeline pass retrieves five revision-pinned shards containing 25,000 structures in total. FoldPipe directly recorded 16.33 s mean overlap between shard retrieval and training, compared with 0 s by construction for the sequential bounded baseline. Mean pass time was 76.78 s for FoldPipe and 83.37 s for the baseline. However, the primary paired estimate was only $1.0587\times$ geometric-mean speedup with a 95% bootstrap interval of $0.8776\times$ – $1.2880\times$; intervals for paired time savings also crossed zero. The experiment therefore demonstrates operation of the overlap mechanism but is inconclusive about a speed advantage under the observed public-network variability. This negative statistical result delineates the system’s useful regime: prefetch can hide only the portion of retrieval that coincides with current-shard computation.

1 Introduction

Molecular machine-learning workloads commonly represent each conformation as a small graph carrying atomic numbers, coordinates, energies, and forces. Model implementations such as SchNet [1] can consume these objects through PyTorch Geometric (PyG) [2], but dataset preparation and deployment choices still determine whether an accelerator receives batches on time. On a temporary notebook runner, a processed corpus may live in remote object storage rather than on a fast local filesystem, while the complete corpus may be larger than the runner’s memory or persistent disk budget.

General-purpose solutions already cover much of this design space. PyTorch provides multiprocessing data loading [3]; PyG provides database-backed on-disk datasets and host-to-device prefetch utilities [4, 5]; Hugging Face provides revision-addressable remote repository storage and file retrieval [6]; and WebDataset uses streaming tar shards [7]. Larger systems optimize preprocessing, caching, and out-of-core graph access at substantially greater scope [8–11]. FoldPipe does not replace these systems.

Instead, FoldPipe targets a narrower migration problem: a researcher already has bounded-size `.pt` shards containing native tensor or PyG objects and wants to consume them directly from Hugging Face or Google Drive without first re-encoding them into tar records or a database. FoldPipe adds a two-method source interface and a one-stage producer–consumer loop around those files. Its scientific value depends less on algorithmic novelty than on stating the performance boundary correctly and publishing enough timing provenance to test it.

This paper makes three contributions:

1. It describes a small, backend-independent integration for remote, native PyTorch/PyG shards whose number of live shard payloads is constant in the total dataset size.

2. It provides a traceable paired benchmark protocol that records source and data revisions, pipeline order, per-shard transfer and training intervals, payload bytes, memory, utilization samples, and raw paired outcomes.
3. It reports a scientifically conservative real-network result: overlap is observed, but a throughput advantage is not statistically established in 20 paired MD17/SchNet passes.

No claim is made that asynchronous prefetch, bounded buffering, or the ideal two-stage overlap equation is novel.

2 Background and Related Work

2.1 Prefetch and ML input pipelines

Overlapping input with computation predates contemporary machine learning. Shriver, Small, and Smith analyzed why filesystem prefetch works and explicitly identified computation-I/O overlap as one source of throughput improvement [12]. The familiar result that equal-duration stages can approach a twofold throughput improvement is therefore prior art, not a FoldPipe contribution.

Modern ML frameworks broaden the same principle. The `tf.data` runtime composes parallelism, caching, and input transformations while overlapping communication and computation [8]. CoordDL analyzes fetch and preprocessing stalls and coordinates preprocessing and caching [9]. FFCV combines a dedicated format, caching, preloading, asynchronous transfer, and compilation [13]. OneAccess argues for cluster-wide sharing across jobs [14], while Cachew adds managed resource scaling and caching [10]. These systems pursue throughput, sharing, and automatic optimization beyond FoldPipe’s scope.

2.2 Formats, remote streaming, and graph systems

WebDataset packages samples into tar shards designed for sequential streaming from local or remote storage [7]. Hugging Face Datasets offers iterable streaming, sharding, and buffered shuffle for supported dataset formats [6]. FoldPipe’s distinction is compatibility, not superior generality: it accepts already-produced `.pt` shard objects without a separate format migration. This also gives up capabilities such as sample-level streaming within a shard, rich transformation graphs, automatic shuffle, and mature distributed partitioning.

PyG’s `OnDiskDataset` stores graph objects through a database backend when they do not fit in CPU memory [4], and its `PrefetchLoader` asynchronously transfers prepared batches from host to device [5]. FoldPipe acts at a different boundary: remote file retrieval and deserialization before PyG batching. For single massive graphs, MariusGNN pipelines across the memory/storage hierarchy [11], and Helios uses GPU-initiated asynchronous SSD I/O and heterogeneous caching [15]. FoldPipe is not a competitor to those graph-partitioning systems; its shards contain many independent small molecular graphs.

2.3 Molecular workload

The evaluation uses the aspirin trajectory from MD17, introduced with gradient-domain molecular force fields [16], transformed by PyG into graph objects. SchNet’s continuous-filter architecture provides the energy model used in each timed optimization step [1]. The paper evaluates data-path behavior, not a new molecular representation, force field, or accuracy result.

3 FoldPipe Design

3.1 Source abstraction

A `Source` exposes two operations: a lazy iterator over shard identifiers and a function that retrieves and deserializes one identifier. The release contains Hugging Face, Google Drive, controlled-latency, and pre-enumerated sources. Backend discovery is separated from consumption so a benchmark may freeze its shard list before starting a timed pass.

The Hugging Face implementation requests a revision-specific file, streams the response into an in-memory byte buffer, and deserializes it on the CPU. This avoids a local-disk cache and a separate dataset-format conversion.

It is not zero-copy: the serialized bytes and deserialized objects can coexist briefly. Moreover, the current implementation calls `torch.load` with `weights_only=False`; shards must therefore come from a trusted source.

3.2 One-stage prefetch

The consumer submits retrieval of the first shard to a single-worker thread pool. After waiting for shard i , it immediately submits shard $i + 1$ and then yields batches from shard i to the training loop. When the current shard is exhausted, the consumer waits only for any unfinished portion of the next future.

For n shards, let D_i be retrieval plus deserialization time and C_i the time consuming shard i . Ignoring small orchestration overheads, the sequential bounded path has

$$T_{\text{seq}} = \sum_{i=1}^n (D_i + C_i), \quad (1)$$

whereas an ideal one-stage pipeline has

$$T_{\text{pipe}} = D_1 + \sum_{i=1}^{n-1} \max(C_i, D_{i+1}) + C_n. \quad (2)$$

In a homogeneous long stream this approaches per-shard time $\max(D, C)$ instead of $D + C$. The model explains two important limits. First, startup and drain cannot be hidden. Second, if retrieval is much slower than computation, only C of each later retrieval can be masked. These are standard pipeline properties [8, 12], included to interpret measurements rather than claimed as a new result.

3.3 Memory invariant

Shard identifiers are produced lazily. At steady state, the main thread can hold the current deserialized shard while the future holds the next shard or its in-progress byte buffer. Mini-batch objects add a smaller workload-specific term. If every shard is bounded by S_{max} , the live data-path memory is $O(S_{\text{max}})$ and independent of the total shard count, with a constant factor that can approach two shard payloads. FoldPipe therefore does not promise “one shard exactly,” zero memory, or immunity to out-of-memory failures from an individually oversized shard.

4 Experimental Method

4.1 Workload and environment

The released experiment ran on a Kaggle Tesla T4. It retrieved five private Hugging Face shards derived from PyG’s MD17 `aspirin` dataset. Each shard contains 5,000 graph objects, so one timed pass processes 25,000 structures. The shard list was fixed before timing and the dataset repository was pinned to revision `f779686deb9217877dd7dde99b2522bd441492a`.

The model was PyG SchNet with 128 hidden channels, 128 filters, six interaction blocks, 50 Gaussian basis functions, and a 10 Å cutoff. A batch contains 32 molecular graphs. Each step predicts energy, obtains forces as the negative position gradient, and optimizes an energy mean-squared error plus ten times the force mean-squared error. The model received one untimed warm-up batch.

4.2 Paired protocol

The experiment formed 20 pairs. Within each pair, both the sequential bounded stream and FoldPipe processed the same five shards from the same initial model state and manual seed. Execution order alternated by pair to distribute time-varying network conditions: sequential then FoldPipe for even-indexed pairs, and the reverse for odd-indexed pairs. This yielded 40 timed passes, 200 shard traces, and 1,000,000 repeated structure-visits. The last quantity must not be interpreted as one million unique structures.

The benchmark used monotonic timestamps for download start, download finish, deserialization finish, training start, and training finish. Resident set size and GPU utilization from `nvidia-smi` were sampled every 0.5 s. GPU synchronization delimited each shard’s training interval. Discovery overhead was excluded by the fixed shard list, while each timed pass still retrieved and deserialized every payload.

Table 1: Twenty-pair MD17/SchNet summary. Pipeline-specific means are descriptive; inference about runtime uses the paired effects below.

Metric	Sequential	FoldPipe
Mean pass time (s)	83.37	76.78
Median pass time (s)	75.27	75.97
Time standard deviation (s)	39.86	28.46
Mean sampled GPU utilization (%)	36.17	37.70
Mean peak RSS (GiB)	1.935	2.064
Mean I/O–compute overlap (s)	0.00	16.33
Mean measured GPU wait (s)	56.50	48.65

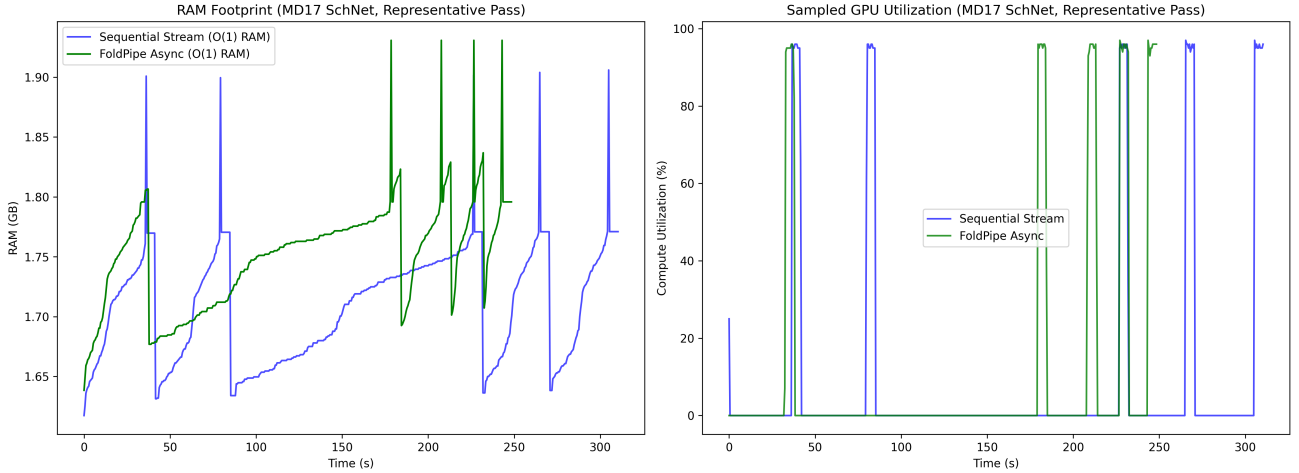


Figure 1: Resident memory and sampled GPU utilization for the first recorded pass of each pipeline. These representative traces illustrate bursty execution but are not the basis of the paired runtime inference.

The primary multiplicative estimand is the geometric mean of the 20 paired ratios $T_{\text{seq}}/T_{\text{FoldPipe}}$. Its 95% interval is a deterministic percentile bootstrap of paired log ratios with 20,000 resamples. Paired arithmetic time savings and their median are secondary estimands with paired percentile-bootstrap intervals. Pipeline-specific means are reported descriptively. The full raw traces and seeds are included in the v0.3.0 artifact [17, 18].

The executed source tree was clean commit `16fdbb26b00f9721ce4034335ce0ee12bda77720`. A source manifest records a bundle SHA-256 of `4fc3f6d712557822efc0d5d71aaa616b0fe30f578631726ef60d6a0fe73b32c0`.

5 Results

FoldPipe was faster in 11 of 20 pairs. The geometric mean paired speedup was $1.0587\times$, but its 95% interval was $0.8776\times$ – $1.2880\times$. Mean paired time saved was 6.59 s with a 95% interval of -7.69 to 21.11 s. Median paired time saved was 3.45 s with an interval of -13.48 to 22.89 s. All three intervals include their respective no-effect values, so this experiment is inconclusive about a wall-clock speed advantage.

The mechanism measurement answers a different question. FoldPipe recorded 16.33 s mean overlap per pass (95% pipeline-specific bootstrap interval 15.04–17.70 s), whereas the sequential implementation has no concurrent retrieval and training by construction. Thus the implementation did execute the intended overlap. That overlap was insufficient to overcome large between-pass network variation reliably in this sample.

Peak RSS was higher for FoldPipe by 0.129 GiB in the unpaired mean summary, which is consistent with retaining a prefetched next payload. The experiment does not support a claim that asynchronous execution uses less memory than the sequential bounded stream; both hold a constant number of shards. Likewise, the 1.53 percentage-point difference in sampled mean GPU utilization is treated as descriptive rather than statistically established.

6 Discussion

The result separates mechanism from outcome. A completed future proves that some retrieval occurred during current-shard training; it does not guarantee a shorter pass when D_i varies sharply across a public network. FoldPipe’s benefit should be largest when current-shard computation is comparable to the next retrieval, when enough of D_{i+1} can be hidden without leaving the consumer waiting. In an I/O-dominant regime, a depth-one queue cannot conceal the excess retrieval time. In a compute-dominant regime, retrieval can be mostly hidden, but the total gain is then bounded by a relatively small I/O fraction.

The engineering trade-off is similarly conditional. Researchers who already possess native, trusted `.pt` shards can avoid a conversion stage and retain PyG object semantics. Researchers optimizing stable cluster-scale throughput may prefer WebDataset, FFCV, a database-backed PyG dataset, or a storage-aware graph system. FoldPipe currently offers neither their breadth nor their optimization depth.

7 Limitations and Threats to Validity

Single environment and dataset slice. The real experiment uses one Kaggle T4, one model configuration, the first five aspirin shards, and one public remote service. It does not establish results for other accelerators, molecules, regions, object stores, or heavier equivariant models.

Repeated and temporally correlated observations. The same 25,000 structures are revisited in every pass. Network conditions can remain correlated across adjacent pairs, while the percentile bootstrap treats pairs as exchangeable. Twenty pairs leave a wide interval. Alternating order reduces a simple first-run bias but is not full randomization or a time-series model.

Telemetry resolution. GPU utilization is a coarse 0.5 s sample from `nvidia-smi`, not a kernel trace. Process RSS omits some external allocations and does not prove an asymptotic memory bound empirically. The bound follows from the live-object invariant under a bounded maximum shard size.

Pipeline scope. There is one prefetch worker and no adaptive queue depth, local persistent cache, retry/checkpoint protocol, distributed partitioning, random access, or sample-level shuffle. A very large shard can still exhaust memory. Python object deserialization may be significant and is unsafe for untrusted artifacts.

Scientific scope. The benchmark times optimization steps but does not report validation error, force accuracy, convergence, or simulation stability. It is a systems benchmark over an established molecular workload, not evidence of a better molecular potential.

Dataset metadata. The private Hugging Face mirror is revision-pinned, but at the time of writing it lacks a dataset card and explicit license metadata. The repository’s migration script traces it to PyG’s original MD17 aspirin loader; nevertheless, the mirror must receive an upstream citation, transformation description, and verified redistribution license before public artifact release or manuscript submission.

8 Reproducibility and Research Integrity

The GitHub release contains raw JSON for every pair and shard, a generated report, plot, execution log, and source manifest. The Kaggle artifact records the exact data revision and executed code bundle. The repository is MIT licensed [17]. The current software release is FoldPipe 0.3.2; the MD17 measurements analyzed here are preserved in the frozen v0.3.0 research artifact.

The manuscript deliberately attributes prefetching and input pipelining to prior work, uses no third-party figure, and reports the inconclusive paired intervals alongside point estimates. A separate repository audit documents public-code phrase searches, commit provenance, claim boundaries, and remaining dataset-license work. Before submission, the rendered manuscript should also be checked with the target institution’s publisher-corpus similarity service; public-web searches cannot provide equivalent coverage.

9 Conclusion

FoldPipe is a compact adapter for a specific deployment path: remote, already-sharded, native molecular graph objects on constrained compute. Its one-stage background fetch creates measurable I/O-compute overlap while keeping the number of live shards independent of total dataset size. In the released 20-pair MD17/SchNet study, the mechanism operated as designed, but network variability left the runtime effect unresolved. The defensible claim is therefore conditional: FoldPipe can hide the portion of next-shard retrieval concurrent with current-shard computation; it does not guarantee acceleration. Broader performance claims require controlled multi-seed crossover experiments and additional real environments.

References

- [1] Kristof T. Schütt, Huziel E. Sauceda, Pieter-Jan Kindermans, Alexandre Tkatchenko, and Klaus-Robert Müller. Schnet—a deep learning architecture for molecules and materials. *The Journal of Chemical Physics*, 148(24):241722, 2018. doi: 10.1063/1.5019779. URL <https://doi.org/10.1063/1.5019779>.
- [2] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019. doi: 10.48550/arXiv.1903.02428. URL <https://arxiv.org/abs/1903.02428>.
- [3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019. URL <https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>.
- [4] PyTorch Geometric Contributors. Ondiskdataset documentation. PyTorch Geometric documentation, 2026. URL https://pytorch-geometric.readthedocs.io/en/stable/generated/torch_geometric.data.OnDiskDataset.html. Accessed 2026-08-17.
- [5] PyTorch Geometric Contributors. Loader documentation: Prefetchloader. PyTorch Geometric documentation, 2026. URL <https://pytorch-geometric.readthedocs.io/en/latest/modules/loader.html>. Accessed 2026-08-17.
- [6] Hugging Face. Downloading files from the hugging face hub. Hugging Face Hub Documentation, 2026. URL https://huggingface.co/docs/huggingface_hub/guides/download. Accessed 2026-08-20.
- [7] Alex Aizman, Gavin Maltby, and Thomas Breuel. High performance i/o for large scale deep learning. *arXiv preprint arXiv:2001.01858*, 2020. doi: 10.48550/arXiv.2001.01858. URL <https://arxiv.org/abs/2001.01858>.
- [8] Derek G. Murray, Jiri Simsa, Ana Klimovic, and Ihor Indyk. tf.data: A machine learning data processing framework. *Proceedings of the VLDB Endowment*, 14(12):2945–2958, 2021. doi: 10.14778/3476311.3476374. URL <https://arxiv.org/abs/2101.12127>.
- [9] Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. Analyzing and mitigating data stalls in dnn training. *Proceedings of the VLDB Endowment*, 14(5):771–784, 2021. doi: 10.14778/3446095.3446100. URL <https://www.vldb.org/pvldb/vol14/p771-mohan.pdf>.
- [10] Dan Graur, Damien Aymon, Dan Kluser, Tanguy Albrici, Chandramohan A. Thekkath, and Ana Klimovic. Cachew: Machine learning input data processing as a service. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 689–706. USENIX Association, 2022. URL <https://www.usenix.org/conference/atc22/presentation/graur>.
- [11] Roger Waleffe, Jason Mohoney, Theodoros Rekatsinas, and Shivaram Venkataraman. Mariusgnn: Resource-efficient out-of-core training of graph neural networks. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages 144–161, 2023. doi: 10.1145/3552326.3567501. URL <https://doi.org/10.1145/3552326.3567501>.

- [12] Elizabeth Shriver, Christopher Small, and Keith A. Smith. Why does file system prefetching work? In *1999 USENIX Annual Technical Conference*, pages 71–84. USENIX Association, 1999. URL https://www.usenix.org/legacy/publications/library/proceedings/usenix99/full_papers/shriver/shriver_html/index.html.
- [13] Guillaume Leclerc, Andrew Ilyas, Logan Engstrom, Sung Min Park, Hadi Salman, and Aleksander Madry. Ffcv: Accelerating training by removing data bottlenecks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. URL <https://arxiv.org/abs/2306.12517>.
- [14] Aarati Kakaraparthi, Abhay Venkatesh, Amar Phanishayee, and Shivaram Venkataraman. The case for unifying data loading in machine learning clusters. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*. USENIX Association, 2019. URL <https://www.usenix.org/conference/hotcloud19/presentation/kakaraparthi>.
- [15] Jie Sun, Mo Sun, Zheng Zhang, Jun Xie, Zuocheng Shi, Zihan Yang, Jie Zhang, Fei Wu, and Zeke Wang. Helios: An efficient out-of-core gnn training system on terabyte-scale graphs with in-memory performance. *arXiv preprint arXiv:2310.00837*, 2023. doi: 10.48550/arXiv.2310.00837. URL <https://arxiv.org/abs/2310.00837>.
- [16] Stefan Chmiela, Alexandre Tkatchenko, Huziel E. Sauceda, Igor Poltavsky, Kristof T. Schütt, and Klaus-Robert Müller. Machine learning of accurate energy-conserving molecular force fields. *Science Advances*, 3(5):e1603015, 2017. doi: 10.1126/sciadv.1603015. URL <https://doi.org/10.1126/sciadv.1603015>.
- [17] Dhiren Khatri. Foldpipe, version 0.3.0. Software release, 2026. URL <https://github.com/aviator1f/FoldPipe/releases/tag/v0.3.0>.
- [18] Dhiren Khatri. Foldpipe md17 rigorous benchmark, kaggle version 8. Reproducibility artifact, 2026. URL <https://www.kaggle.com/code/dhirenkhatri/foldpipe-md17-rigorous-benchmark>.